Data Structures and Algorithms

Lecture 09

Outlines

- Pre Requisites
  - Recursive Algorithm
- Abstract Data Type: **Tree**

Recursive Algorithms

- Consider a Factorial of number **n**
  - N=5
    - **5!=5.4.3.2.1 = 120**
  - N=6
    - **6!= 6.5.4.3.2.1 = 720**
  - N=3
    - **3!= 3.2.1 = 6**
  - N=0
    - **0! = 1**

Recursive Algorithm

- An iterative algorithm to solve factorial problem

```
function factorial( int number) {
   int result=1;
   for( int i=1;i<=number;i++) {
        result*=i;
   }
   print result
}
```

n * fact ((n-1))

Recursive Algorithms

- Another way to solve factorial problem is to use iterative algorithm

```
print factorial(5);
function factorial(int number)
{
    if(number==1)
        return 1;
     return number * factorial(number-1);
}
```

Outline

- Abstract Data Type: **Tree**
  - Definition of Tree
  - Terminologies
  - Binary Tree (B-Tree)
  - Unbalanced Tree
  - Representation of Tree
  - Tree Operations:
    - Traversal
      - In-order
      - Pre-order
      - Post-order

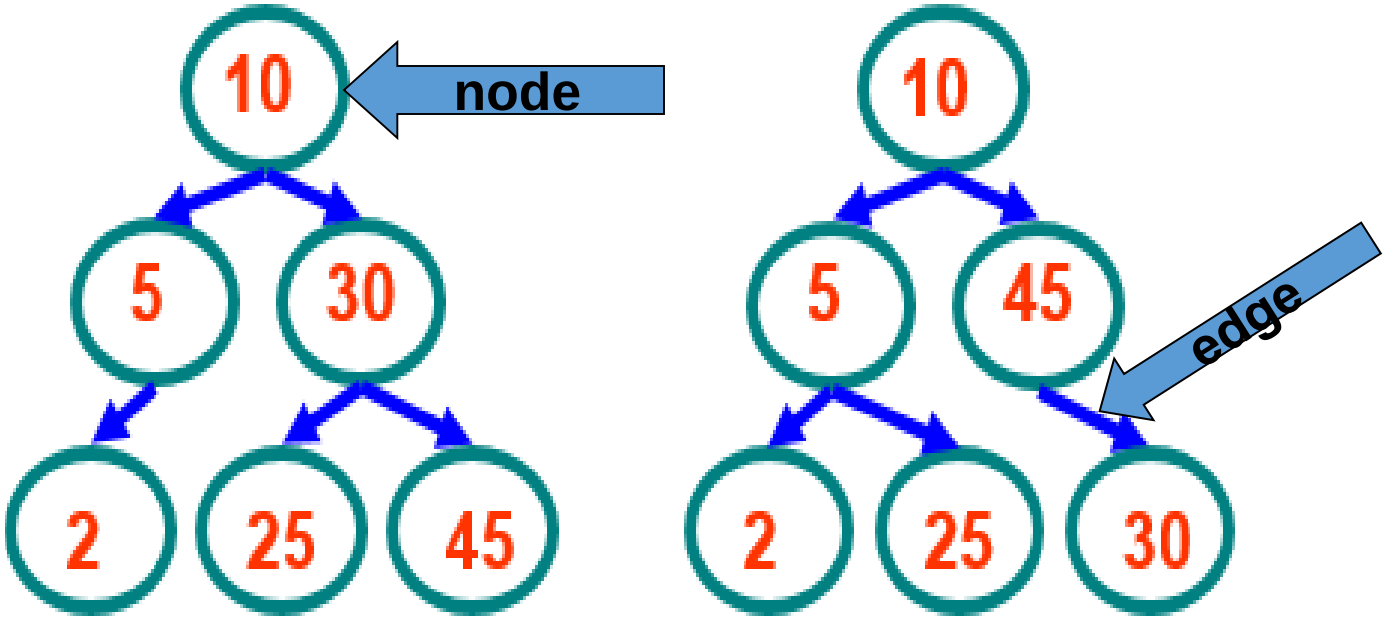
Tree

- Definition
  - Fundamental Data Storage Structure in Programming
  - Combines Advantages of
    - sorted Array
      - Searching as fast as in sorted array (binary search)
  - and
    - Linked List.
      - Insertion and deletion as fast as in linked list.

Tree

- A Tree consist of nodes (presents data e.g. Person, car etc.) and edges(connection between two nodes)
- Example

10,5,30,2,25,45,50



Terminologies

- **Path**: Traversal from node to node along the edges results in a sequence called path.
- **Root**: Node at the top of the tree.
- **Parent**: **will be discussed in upcoming lectures**
- **Child**: Any node may have one or more lines/edges running downward to other nodes. Nodes below are children.
- **Leaf**: A node that has no children.
- **Subtree**: Any node, that is not a root node, which consist of child and possibly its child have other child can be considered subtree.

Terminologies

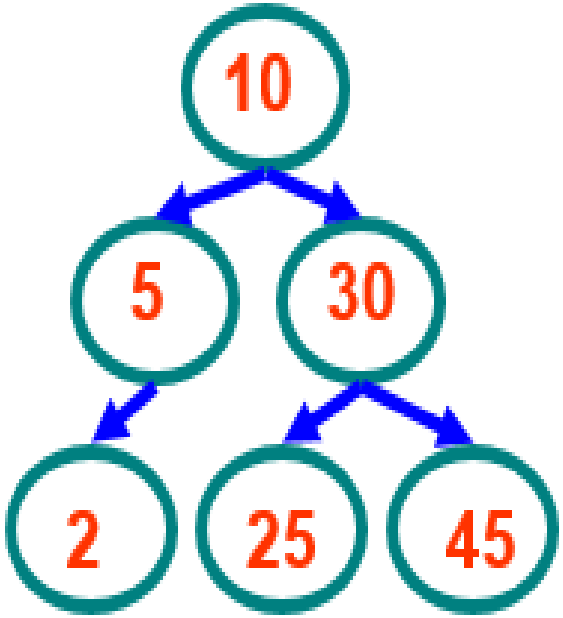
- **Visiting**: A node is visited when program control arrives at the node, usually for processing.
- **Traversing**: To traverse a tree means to visit all the nodes in some specified order.
- **Levels**: The level of a particular node refers to how many generations the node is from the root. Root is assumed to be level 0.
- **Keys**: Key value is used to search for the item or perform other operations on it.

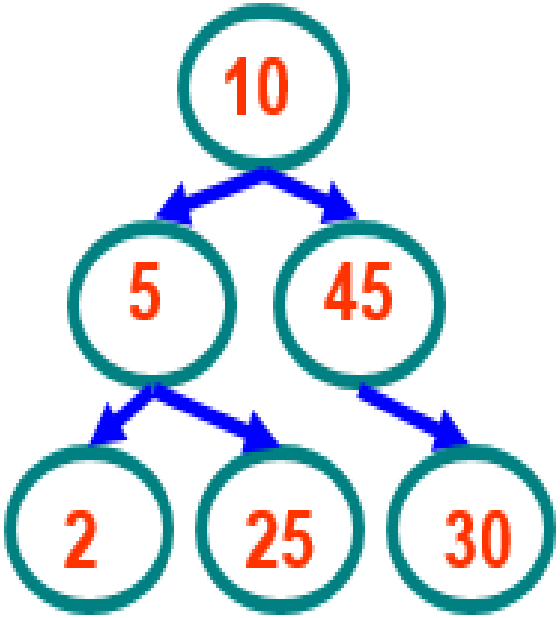
Terminologies

Level 0

Level 1

Level 2

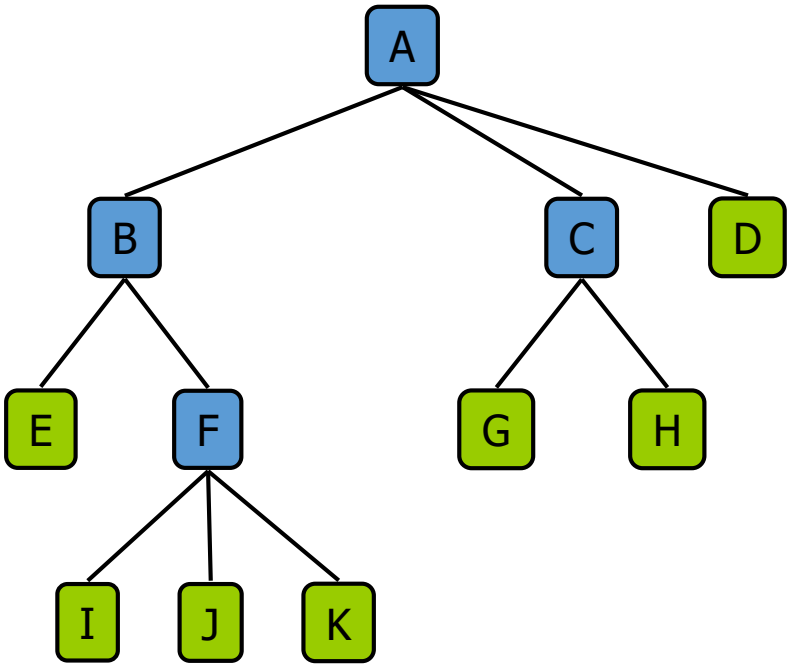




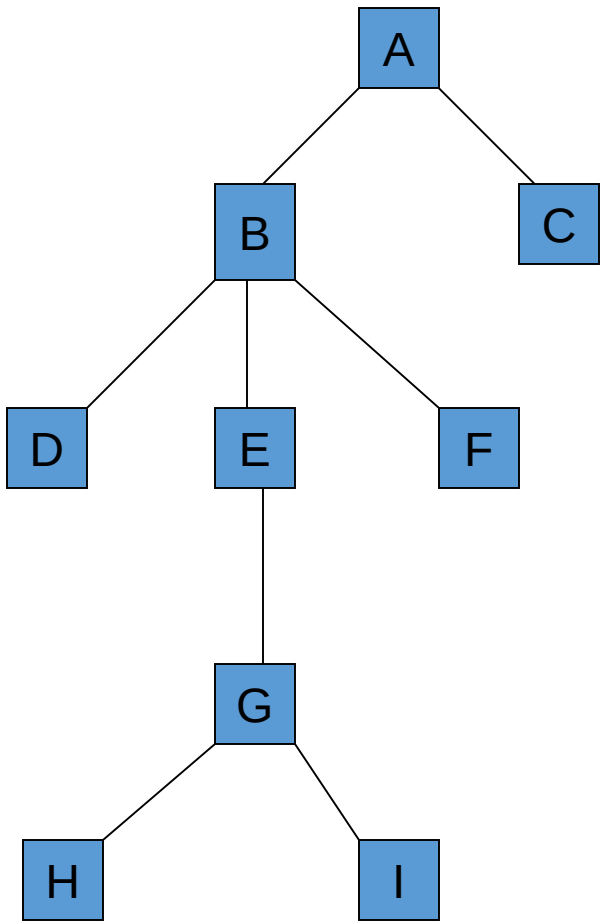
Terminologies

- **Root**: node without parent (A)
- **Siblings**: nodes share the same parent
- **Internal node**: node with at least one child (A, B, C, F)
- **External node** (leaf): node without children (E, I, J, K, G, H, D)
- **Ancestors** of a node: parent, grandparent, grand-grandparent, etc.
- **Descendant** of a node: child, grandchild, grand-grandchild, etc.
- **Depth** of a node: number of ancestors
- **Height** of a tree: maximum depth of any node (3)
- **Degree** of a node: the number of its children
- **Degree** of a tree: the maximum number of its node.

- **Subtree**: tree consisting of a node and its descendants



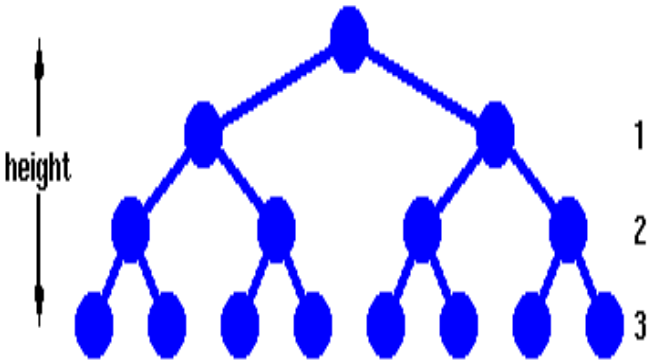
Terminologies



Property	Value
Number of nodes	9
Height	4
Root Node	A
Leaves	D,H,I,F,C
Interior nodes	A,B,E,G
Ancestors of H	A,B,E,G
Descendants of B	E,G,H,I,D,F
Siblings of E	D,F
Right subtree of A	C
Degree of this tree	9
Dept of G	3

Binary Tree (B-Tree)

- A Tree where every node can have at most **two children**.
- The two children of each node are called the left child and right child corresponding to their positions.
- A node can have only a left child or only a right child or it can have no children at all.
- Left child is always less that its parent, while right child is greater than its parent.



Unbalanced Tree

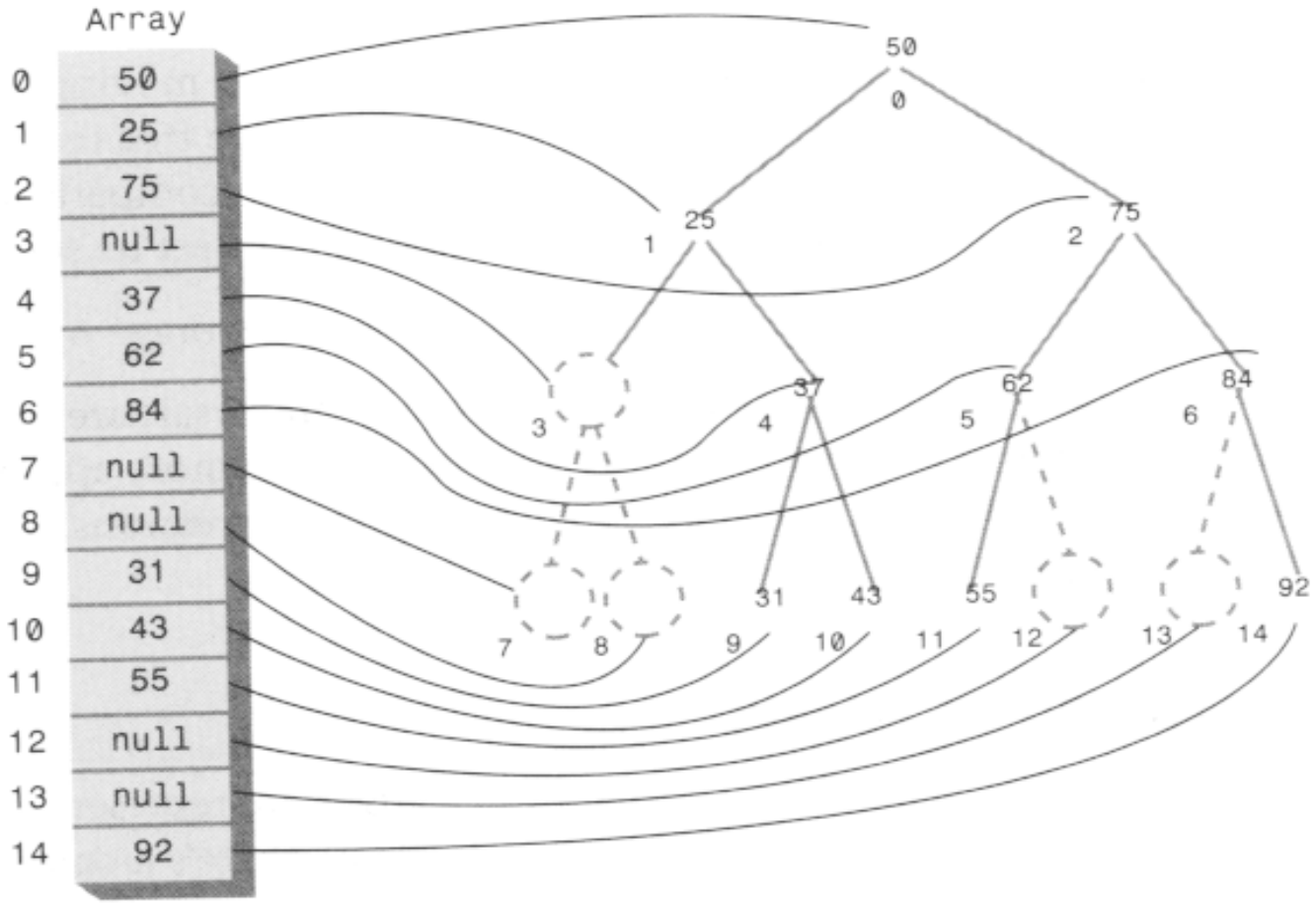
- Some trees can be unbalanced.
- They have most of their nodes on one side of the root or the other. Individual subtrees may also be unbalanced.
- Trees become unbalanced because of the order in which the data items are inserted.
- If the key values are inserted in ascending or descending order the tree will be unbalanced.
- For search-centric application (Binary tree), an unbalanced tree must be re-balanced.

Representation of Tree

- A Tree can be based upon
  - Array
    - Parents and Childs are stored at specific position in array.
- OR
  - Linked List
    - Store the nodes at unrelated locations in memory and connect them with other nodes (child nodes) using references

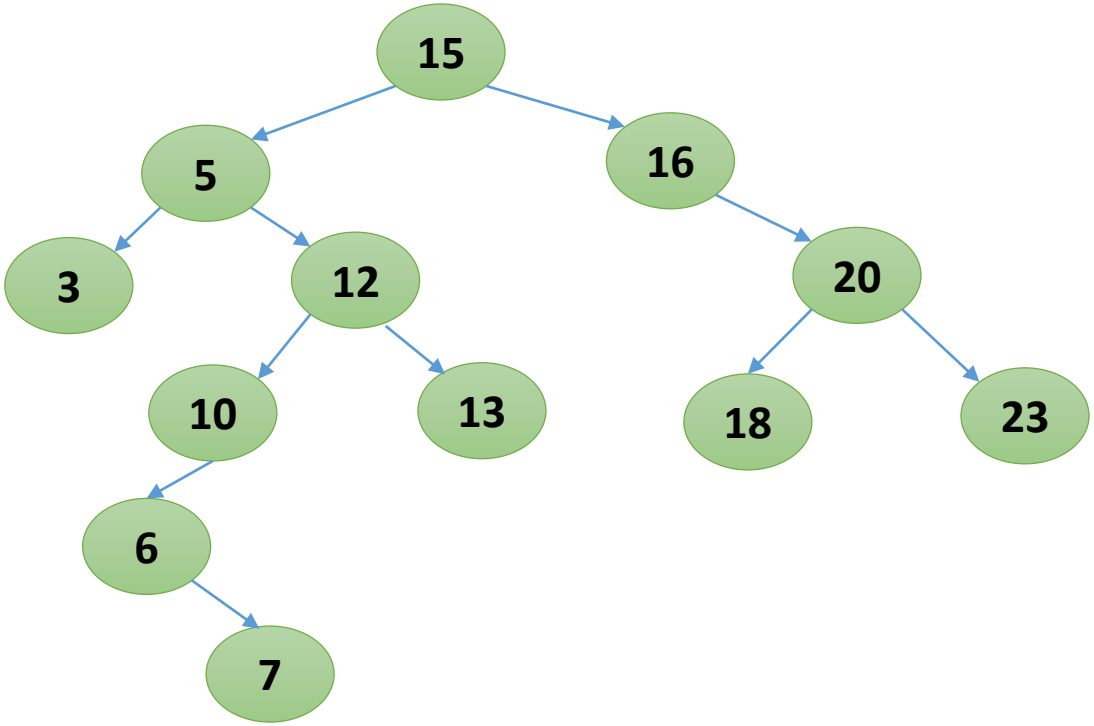
Representation of Tree

- Tree representation by array



Tree Operations

- **Insertion**
  - Insert following elements to Tree:
    - 15, 5, 3, 12, 10, 6, 7,13, 16, 20, 18, 23

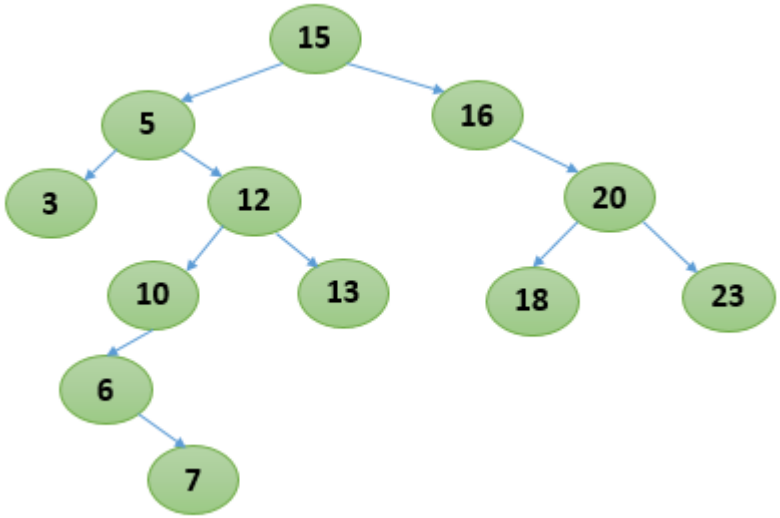


Tree Operations

- **Traversal**
  - Three Simple way to Traverse the tree
    - In-order (left, root, right)
    - Pre-Order (root, left, right)
    - Post-Order (left, right, root)

Tree Operations

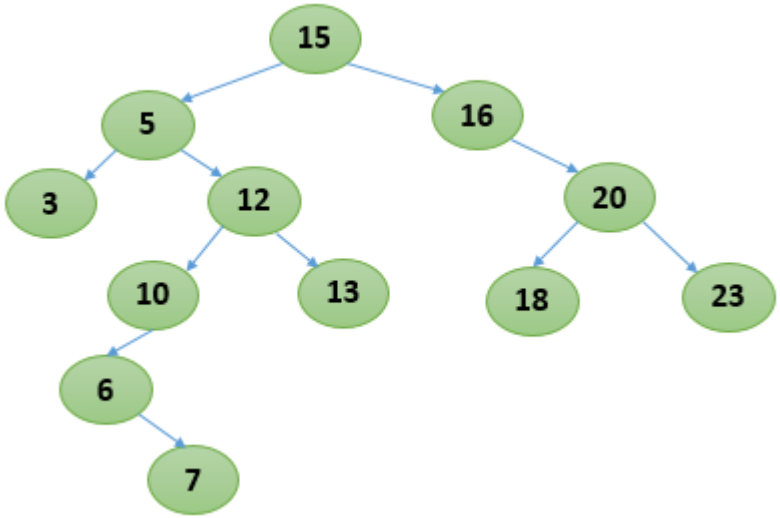
- **Traversal**
  - **In-Order** (left, root, right)



Output:
3,5,6,7,10,12,13,15,16,18,20,23

Tree Operations

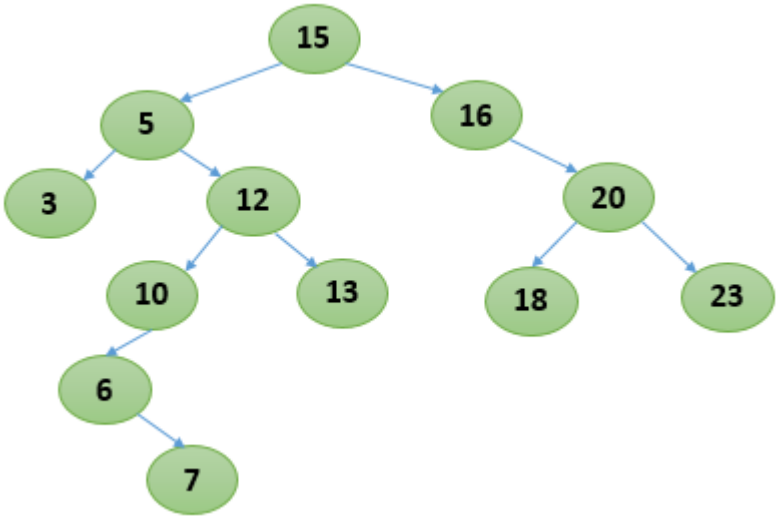
- **Traversal**
  - **Pre-Order** (root, left, right)



Output:
15, 5, 3, 12, 10, 6, 7,13, 16, 20, 18, 23

Tree Operations

- **Traversal**
  - **Post-Order** (left, right, root)



Output:
3, 7, 6, 10, 13, 12, 5, 18, 23, 20, 16, 15